

Computer Graphics Support Group of 30 Phys-Math Lyceum



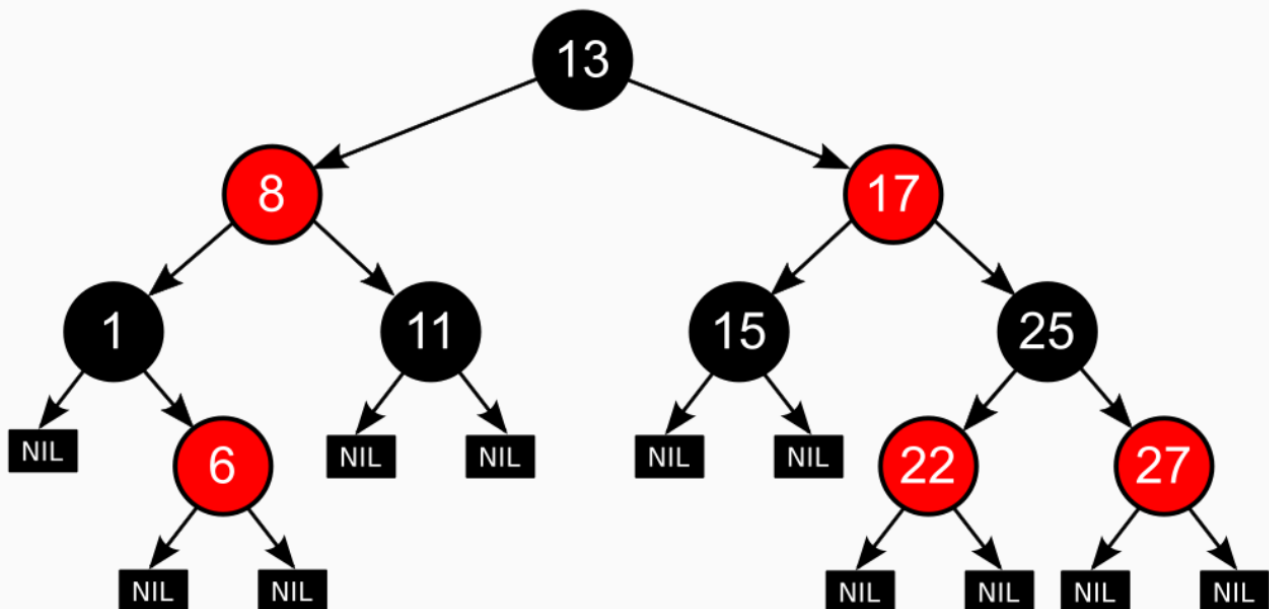
CGSG'SummerCamp'2026, Физико-математический лицей № 30

Red-Black-Trees

Красно-черные деревья

Рудольфа Байера, название Л. Гимбаса и Р. Седжвика (1978).

Красно-чёрное дерево — двоичное дерево поиска, в котором каждый узел имеет атрибут цвета.



Свойства красно-черного дерева:

1. Узел может быть либо красным, либо чёрным и имеет двух потомков.
2. Корень — (как правило) чёрный.
3. Все листья, не содержащие данных — чёрные.
4. Оба потомка каждого красного узла — чёрные.
5. Любой простой путь от узла-предка до листового узла-потомка содержит одинаковое число чёрных узлов.

Листовые узлы красно-чёрных деревьев не содержат данных, благодаря чему не требуют выделения памяти — достаточно записать в узле-предке в качестве указателя на потомка нулевой указатель (NIL).

Количество черных узлов на ветви от корня до листа называется черной высотой дерева. Перечисленные свойства гарантируют, что самая длинная ветвь от корня к листу не более чем вдвое длиннее любой другой ветви от корня к листу. Пусть у нас есть красно-черное дерево. Черная высота равна bh (black height). Если путь от корневого узла до листового содержит минимальное количество красных узлов (т.е. ноль), значит этот путь равен bh . Если же путь содержит максимальное количество красных узлов (bh в соответствии со свойством 4), то этот путь будет равен $2bh$. То есть, пути из корня к листьям могут различаться не более, чем вдвое ($h \leq 2\log(n + 1)$, где h — высота поддерева), этого достаточно, чтобы время выполнения операций в таком дереве было $O(\log(n))$

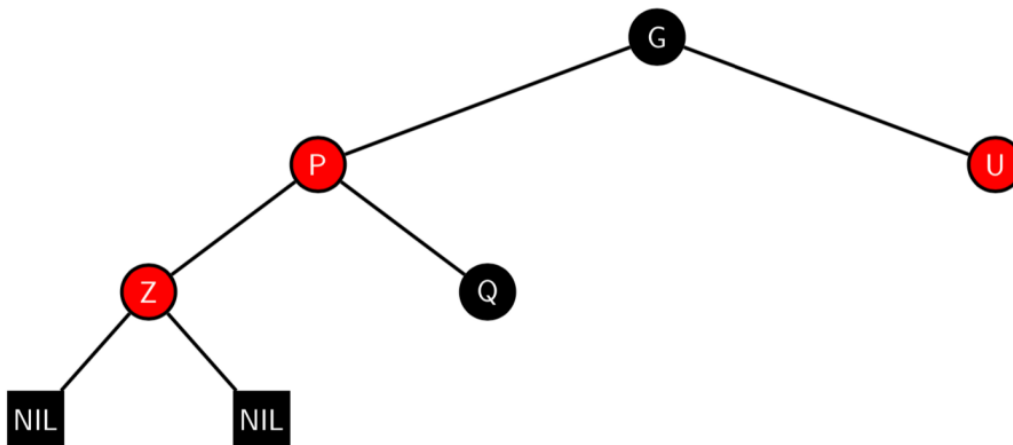
Вставка узла в дерево

Чтобы вставить узел, мы сначала ищем в дереве место, куда его следует добавить. Новый узел всегда добавляется как лист, поэтому оба его потомка являются NIL-узлами и предполагаются черными. После вставки красим узел в красный цвет. После этого смотрим на предка и проверяем, не нарушается ли красно-черное свойство. Если необходимо, мы перекрашиваем узел и производим поворот, чтобы сбалансировать дерево.

1. Добавляем узел красного (R) цвета.
2. Если это корень, то он перекрашивается в черный цвет (B) и свойства не нарушаются.
3. Если родитель для вставляемого узла имеет цвет B, то свойства не нарушаются.
4. Если родитель для вставляемого узла имеет цвет R, то свойства нарушены.

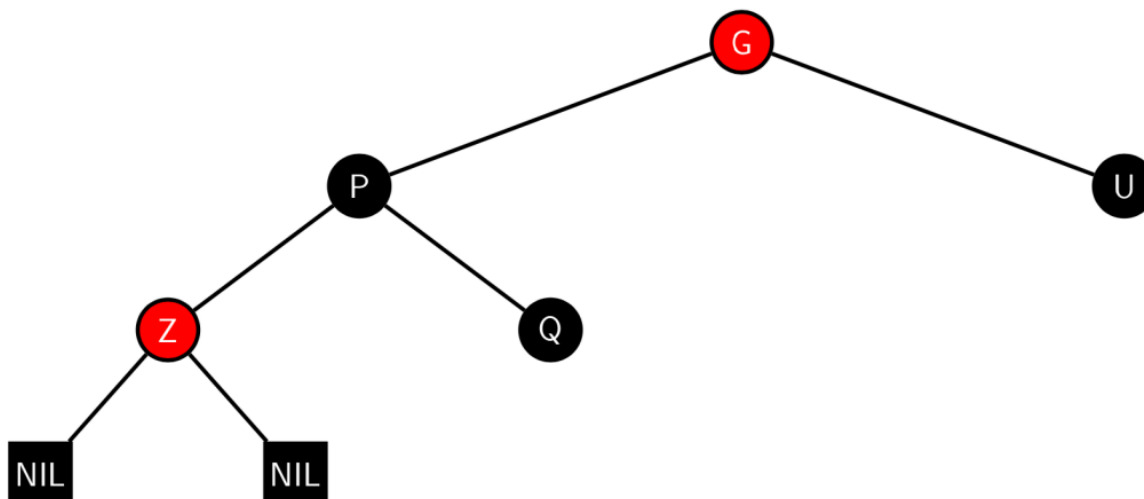
===== Вставка. Случай 1. "Дядя" красный

Мы добавляем в дерево узел Z, который автоматически делается красным.



В данном случае нужно выполнить перекрашивание "родителей" и "деда"

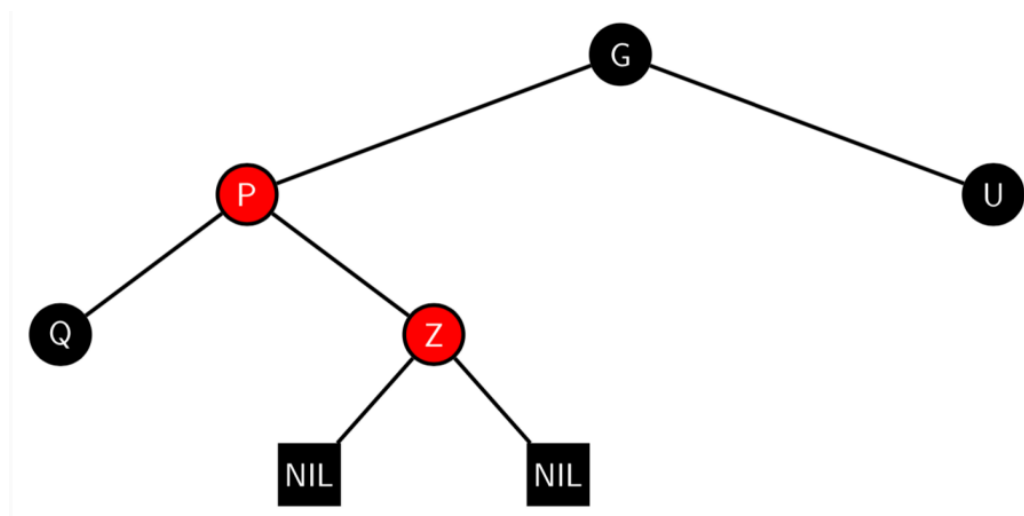
Результат перекрашивания:



Дедушка G теперь может нарушить свойства 2 (Корень — чёрный) или 4 (Оба потомка каждого красного узла — чёрные) (свойство 4 может быть нарушено, так как родитель G может быть красным). Чтобы это исправить, вся процедура рекурсивно выполняется на G.

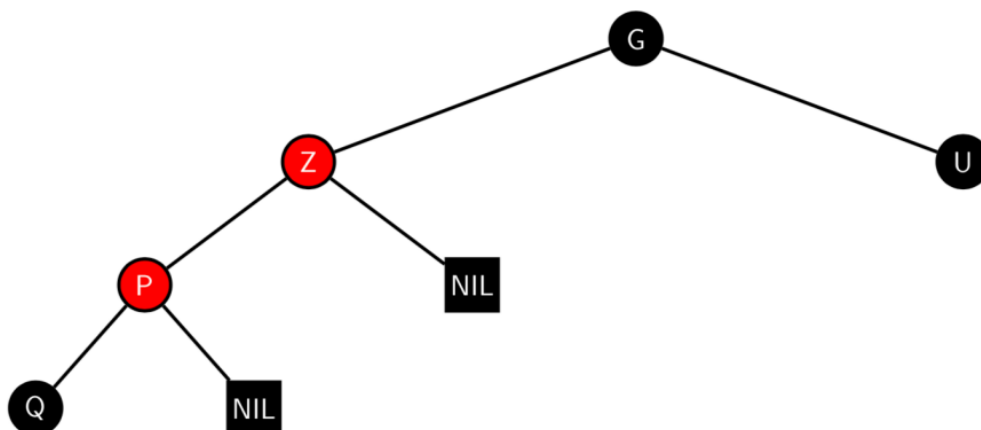
===== Вставка. Случай 2. "Дядя" черный

Дядя черный, добавляемый узел Z - справа от родителя P



В этом случае может быть произведен поворот дерева, который меняет роли текущего узла Z и его предка P.

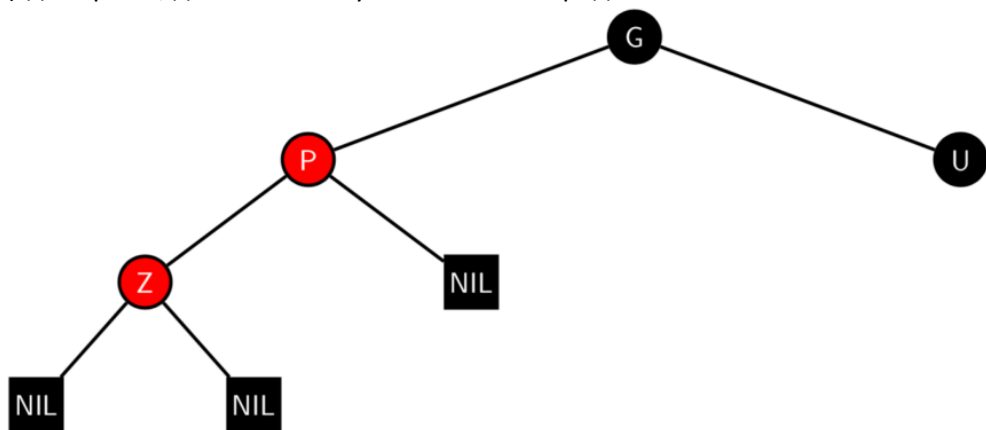
Результат перекрашивания:

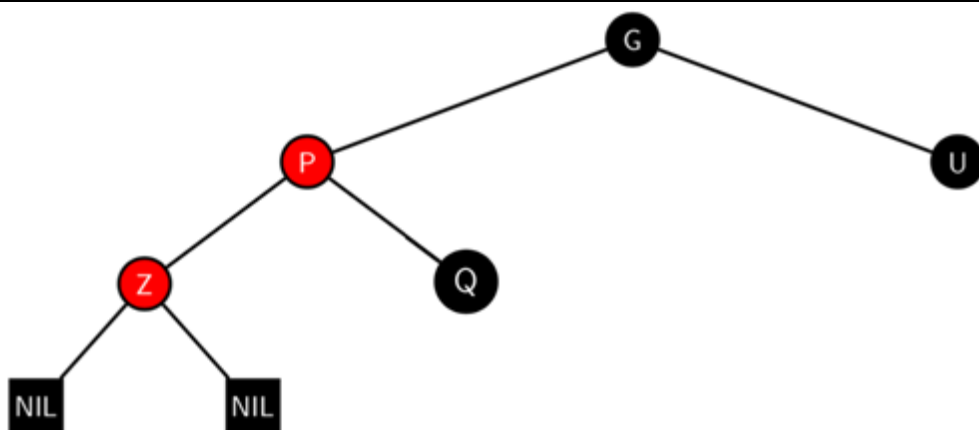


Вращение приводит к тому, что некоторые пути (в поддереве Q на схеме) проходят через узел Z, чего не было до этого. Свойство 4 всё ещё нарушается, но теперь задача сводится к Случаю 3.

===== Случай 3. "Дядя" черный

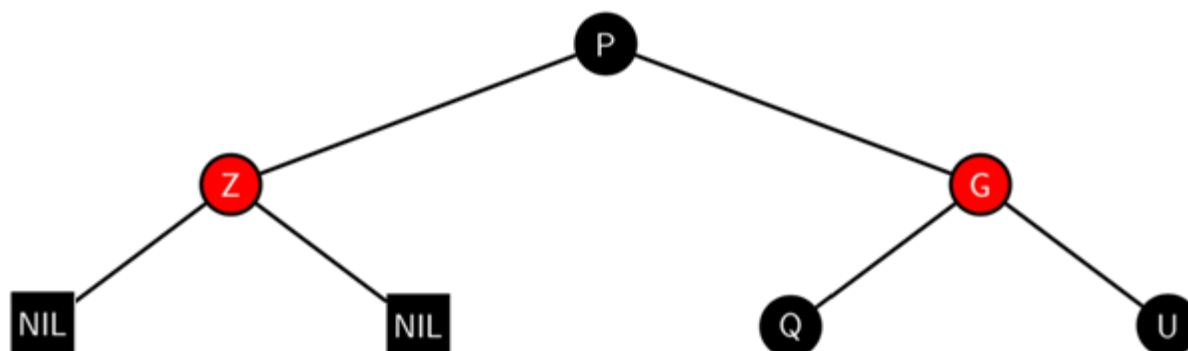
Дядя черный, добавляемый узел Z - слева от родителя P.

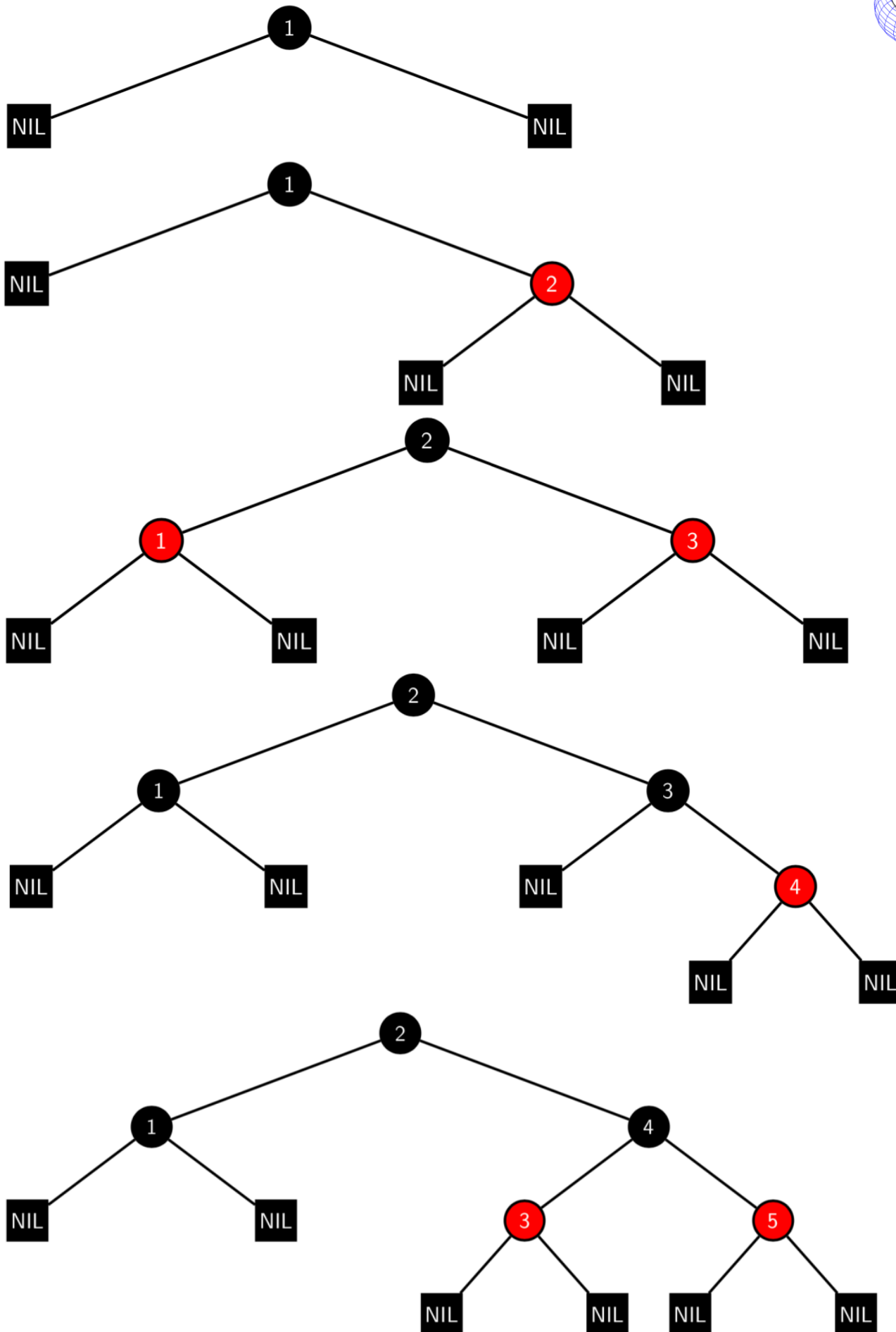


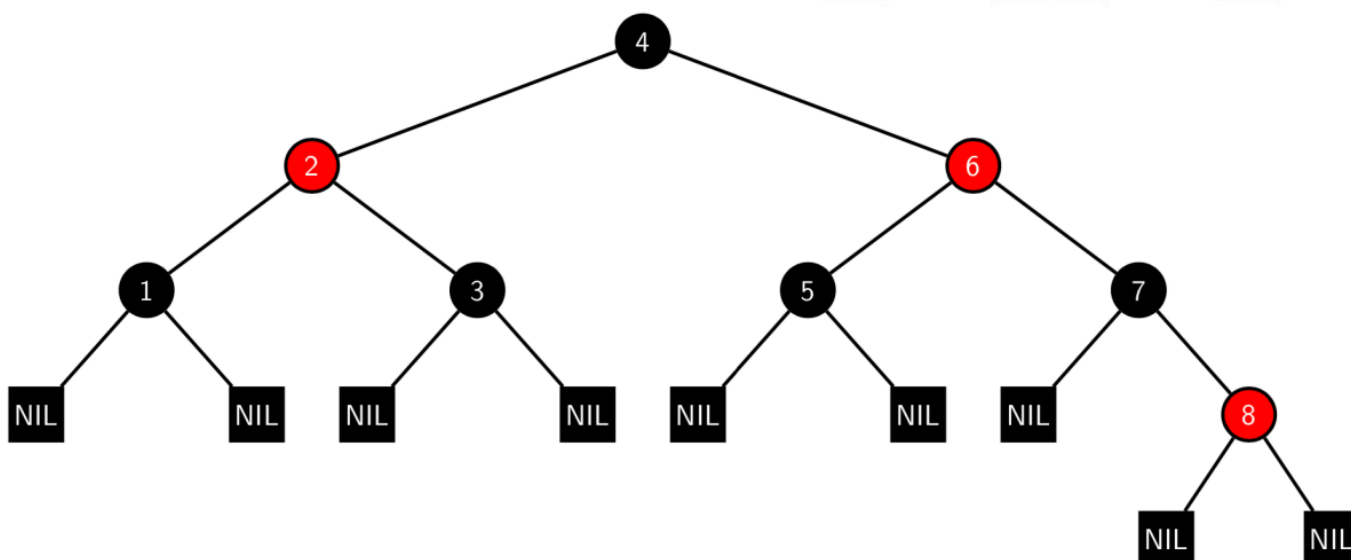
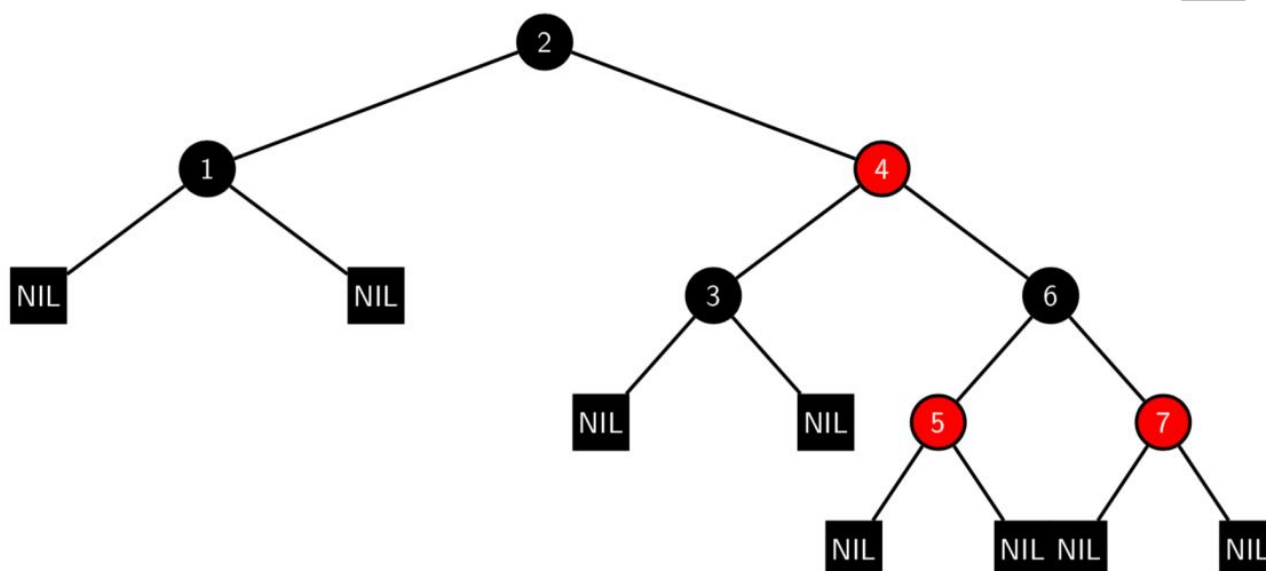
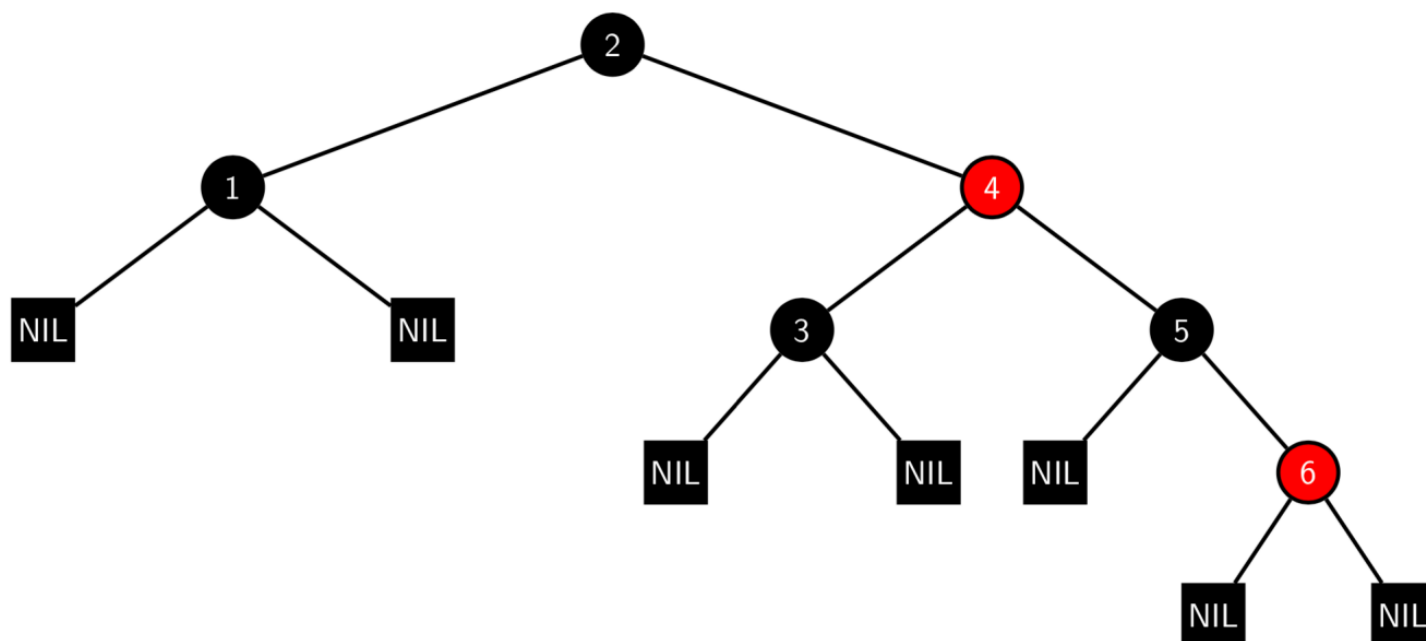


В этом случае выполняется поворот дерева на G. В результате получается дерево, в котором бывший родитель P теперь является родителем и текущего узла Z и бывшего дедушки G.

Результат перекрашивания:









```

/* Red-black tree representation type */
struct rbtree
{
    /* Node color representation type */
    enum rbcolor
    {
        eRED, eBLACK
    }; /* End of 'rbcolor' enum */

    /* Tree node representation type */
    struct node
    {
        INT Key;           // Node key value
        rbcolor Color;     // Node color value
        node
        *Less, *More,      // Node subtree pointers
        *Parent;           // Node parent pointer

        // Node draw info
        DBL
        X, Y,              // Current node position
        XDest, YDest;     // Destination node position

        /* Class constructor.
        * ARGUMENTS:
        *   - new key value:
        *       const key_type &NewKey;
        *   - new data value:
        *       const data_type &NewData;
        *   - new left-right subtree pointer values:
        *       node *NewLeft, *NewRight;
        *   - new parent pointer values:
        *       node *NewParent;
        */
        node( INT NewKey, rbcolor NewColor = eRED,
              node *NewLess = nullptr, node *NewMore = nullptr, node *NewParent = nullptr ) :
            Key(NewKey), Color(NewColor), Less(NewLess), More(NewMore), Parent(NewParent),
            X(0), Y(0), XDest(0), YDest(0)

        {
        } /* End of 'node' function */
    }; /* End of 'node' structure */

    // Tree root pointer
    node *Root;

    /* Class default constructor */
    rbtree( VOID ) : Root(nullptr)
    {
    } /* End of 'rbtree' function */

    /* Class destructor */
    ~rbtree( VOID )
    {
        Clear(&Root);
    } /* End of '~rbtree' function */

    /* Remove copying constructor */
    rbtree( const rbtree & ) = delete;
    /* Remove assignment operator */
    rbtree & operator=( const rbtree & ) = delete;

```



```

/* Clear all tree nodes function.
 * ARGUMENTS:
 *   - pointer to top node pointer:
 *       node **T;
 * RETURNS: None.
 */
VOID Clear( node **T = nullptr )
{
    if (T == nullptr)
        Clear(&Root);
    else
        if (*T != nullptr)
        {
            Clear(&(*T)->Less);
            Clear(&(*T)->More);
            delete *T;
            *T = nullptr;
        }
} /* End of 'ClearTree' function */

/* Check if node color is black function.
 * - node to check:
 *     node *Nd;
 * RETURNS:
 *   (BOOL) TRUE if node color is black, FALSE if it is red one.
 */
BOOL IsBlack( node *Nd )
{
    return Nd == nullptr || Nd->Color == eBLACK;
} /* End of 'IsBlack' function */

/* Rotate node to left (counter clock wise) function.
 * ARGUMENTS:
 *   - node to be rotated:
 *       node *Y;
 * RETURNS: None.
 */
VOID LeftRotate( node *X )
{
    node *Y = X->More;
    X->More = Y->Less;

    if (Y->Less != nullptr)
        Y->Less->Parent = X;

    Y->Parent = X->Parent;

    if (X->Parent == nullptr)
        Root = Y;
    else if (X == X->Parent->Less)
        X->Parent->Less = Y;
    else
        X->Parent->More = Y;

    Y->Less = X;
    X->Parent = Y;
} /* End of 'LeftRotate' function */
. . .

```



```

/* Insert new node function.
 * ARGUMENTS:
 *   - key to be inserted:
 *       INT Key;
 * RETURNS: None.
 */
VOID Insert( INT Key )
{
    node *NewNode = new node(Key), *Parent = nullptr, *TreePtr = Root;

    while (TreePtr != nullptr)
    {
        if (Key == TreePtr->Key)
            return;
        Parent = TreePtr;
        TreePtr = NewNode->Key < TreePtr->Key ? TreePtr->Less : TreePtr->More;
    }
    NewNode->Parent = Parent;
    if (Parent == nullptr)
        Root = NewNode;
    else if (NewNode->Key < Parent->Key)
        Parent->Less = NewNode;
    else
        Parent->More = NewNode;
    CorrectInsert(NewNode);
} /* End of 'Insert' function */

/* Output all tree nodes function.
 * ARGUMENTS: None.
 * RETURNS: None.
 */
VOID Put( VOID )
{
    Put(Root);
    std::cout << std::endl;
} /* End of 'Put' function */

/* Output tree node with subtrees function.
 * ARGUMENTS:
 *   - tree node pointer:
 *       node *T;
 * RETURNS: None.
 */
VOID Put( node *T )
{
    if (T == nullptr)
        std::cout << "\x1b[38;2;55;55;55mNIL\x1b[38;2;255;255;255m";
    else
    {
        std::cout << T->Key <<
            (T->Color == eBLACK ? "\x1b[38;2;55;55;55m[B]" : "\x1b[38;2;255;5;5m[R]") <<
            "\x1b[38;2;255;255;255m\x1b[38;2;255;255;255m";
        std::cout << "(";
        Put(T->Less);
        std::cout << ",";
        Put(T->More);
        std::cout << ")";
    }
} /* End of 'Put' function */
. . .
}; /* End of 'rbtreee' structure */

```